

Phase 3 of 4

Wearable Simulation, Vitals Pipeline & Real-Time Dashboards

Maternal Health Platform — Development Roadmap · Derived strictly from the Complete Data Flow and Original Simulation architecture documents

What This Phase Covers

This phase builds the entire vitals pipeline: the Python simulation engine standing in for a wearable device, the edge-computing layer that decides what to transmit, the MQTT transport layer, the Flask backend processing logic, three-tier MySQL storage, the WebSocket real-time delivery to doctors, and the patient/doctor-facing dashboards that render it. This phase is completely independent of hospital connection (Phase 2) and the LLM (Phase 4).

NOTE: Tech stack for this phase: React + Vite (frontend), Flask (backend, synchronous), MySQL (database, %s placeholders), Mosquitto (self-hosted MQTT broker), and a background worker via Redis RQ.

1. The Simulation Engine — What It Generates

A Python script running as a background worker on a cloud server (Render or Railway) acts as the software equivalent of the patient's wearable IoT device. It generates realistic physiological values for every registered patient whose wearable is in the "Connected" state.

Parameter	Normal Range	Generation Method	Raw Update Frequency
Heart Rate	70–115 bpm	random.randint(70,115)	Every 30 seconds
SpO ₂ (Oxygen)	95–99%	random.randint(95,99)	Every 30 seconds
Blood Glucose (CGM)	70–95 mg/dL fasting	random.uniform(70,95)	Every 60 seconds
Blood Pressure	110–130 / 70–85 mmHg	moving avg of 3 readings locally	Every 3 minutes
Baby Kicks	0–3 per session	random event probability	Once or twice daily
Sleep Quality	Score 0–100	algorithm on HR + motion	Once daily at midnight
Physical Activity/Steps	0–15000 steps	accumulator integer	Once daily at midnight
CO ₂ Level	Normal environmental range	random.uniform(400,600) ppm	Every 30 seconds

The 4-Quadrant State Check (Runs Before Every Packet)

Before generating any value, the simulator checks device state using this priority hierarchy:

- Rule 1 — Table state is supreme: if the device is in table_manual or table_auto state, return vitals=None, status="sensor_off_body".
- Rule 2 — Crisis override (manual beats automatic): if state is crisis_manual, generate emergency values.

- Rule 3 — Autonomous stochastic spike: a 5% dice roll on every cycle can trigger emergency values on its own.
- Default — generate normal healthy values.

2. On-Demand Simulation Lifecycle

Rather than running loops for thousands of offline patients, the system spawns a simulation thread only when a patient is actively connected — a Scale-to-Zero design pattern.

Stage 1 — Connect Event

Patient clicks "Connect Wearable Device" → frontend sends request to backend → backend spawns a dedicated, isolated background thread for that patient → thread begins generating HR, SpO₂, BP, Glucose, etc. → data flows through MQTT → Backend → WebSocket → patient's dashboard shows live values → status changes to "Device connected — live vitals active."

Stage 2 — Concurrency with N Patients

If many patients connect simultaneously, the backend spawns N individual, separate background loops in memory, each tracking its own variables and baseline ranges, running side-by-side without data crossing or overlapping.

Stage 3 — Disconnect Event

Patient clicks "Disconnect Device" → frontend sends a termination request → backend kills that patient's simulation thread completely, freeing memory. Other patients' threads are unaffected.

Stage 4 — Logout (Different From Disconnect)

Logging out of the app is NOT the same as disconnecting the wearable. Logout only destroys the web session — it does not send a kill signal to the Python simulator. The simulation loop stays alive and keeps recording, storing, and streaming vitals to the doctor's dashboard even while the patient is logged out.

Case 1 — Browser Minimized or Closed (Device Still Worn)

The on-demand loop is already running as a background worker on the server; closing the browser tab does not stop it. The doctor's dashboard remains active and uninterrupted; a genuine emergency still flashes a red alarm instantly.

Case 2 — Wearable Placed on a Table

A remote state change (`device_placement = "table"`) intercepts the loop. Instead of generating vitals, it returns Null/Zero placeholders or explicit `sensor_off_body` strings, while the connectivity light stays green (device is powered on, just not worn). The doctor's dashboard shows a flat line or gap with: "Sensor Off-Body / No Signal Detected" — never mistaken for a health crisis.

3. God Mode — Dual-Trigger Anomaly System

Manual Force Spike

A hidden control-panel request (Target: Patient_A | Action: FORCE_SPIKE) updates a `Crisis=True` state for that patient. The simulator reads this on its next cycle and switches to emergency ranges immediately.

Autonomous Stochastic Spike (5% Dice Roll)

Every loop cycle rolls a number 1–100. Landing on 1–5 triggers Crisis Mode automatically. Once triggered, an internal 90-second duration window keeps broadcasting dangerous values before self-correcting back to normal.

Autonomous Table Slippage (2% Dice Roll)

A separate roll: landing on 1–2 simulates the wearable slipping off the wrist unprompted — vitals switch to null/0 while the internet connection stays active.

The 4-Quadrant Matrix

	Intentional (Manual)	Automatic (Stochastic)
Crisis Mode	Quadrant 1 — Forced Spike (manual button)	Quadrant 2 — Autonomous Spike (5% roll, 90s window)
Table Mode	Quadrant 3 — Forced Table State (manual button)	Quadrant 4 — Autonomous Slippage (2% roll)

Priority Hierarchy

- Rule 1 — Table state is supreme: if the device is on a table (Q3 or Q4), it cannot read a pulse — this overrides everything, forcing null data even if a manual crisis spike was also triggered.
- Rule 2 — Manual overrides beat automatic rolls: if the device is being worn, a manual crisis click (Q1) instantly overrides an ongoing autonomous roll (Q2).

4. Edge Layer — Deciding What to Transmit

Runs inside the simulator itself, before any MQTT publish. Pure Python logic, deciding what is worth sending.

Deadband Filter — Heart Rate, SpO₂, Blood Glucose

- Always transmit if a danger zone is crossed (marked CRITICAL)
- Transmit if the change exceeds the deadband threshold (marked CHANGE)
- Force transmit if more than 5 minutes have passed since the last transmission (marked HEARTBEAT)
- Otherwise, suppress — genuinely discarded as a duplicate, not stored anywhere

Thresholds: Heart Rate deadband=10 bpm, danger_low=50, danger_high=120. SpO₂ deadband=2%, danger_low=94. Blood Glucose deadband=5 mg/dL, danger_low=70, danger_high=140.

Moving Window Average — Blood Pressure

Systolic readings above 160 bypass the buffer and transmit immediately as CRITICAL. Otherwise, readings accumulate in a local buffer; after 3 readings (~3 minutes), a smoothed average is published and the buffer clears.

Daily Aggregation — Sleep, Steps, Baby Kicks

Step and kick events accumulate locally through the day. At midnight, one daily_summary packet is published (total_steps, sleep_score, kick_count) and counters reset.

The 5-Minute Heartbeat Rule

Even if nothing changes, one packet is force-transmitted every 5 minutes to confirm the device is alive — this prevents the doctor from thinking the device went offline, and keeps the chart holding a meaningful last value rather than going blank.

5. MQTT Transport Layer

Published using the paho-mqtt Python library to a self-hosted Mosquitto broker over TLS.

Topic Structure

- patients/{patient_id}/vitals/hr, /spo2, /glucose, /bp, /daily
- patients/{patient_id}/status/device
- patients/{patient_id}/alert/critical

QoS Levels

QoS	Meaning	Used For
QoS 0	Fire and forget — fastest, may lose packets	Not used for vitals
QoS 1	At least once delivery	Normal readings
QoS 2	Exactly once, guaranteed delivery	Critical alerts (BP spike, SpO ₂ crash)

Mosquitto Broker

Self-hosted on infrastructure you control (e.g. a small always-on cloud VM). It receives packets, checks its subscriber list, and forwards to matching subscribers. The Flask backend subscribes once using a single wildcard pattern: patients/+ /vitals/+ — the broker routes every patient's packets to the backend, which then sorts by patient_id internally.

6. Flask Backend — Processing and Storage

Receive and Parse

on_message extracts patient_id and param_type from the MQTT topic string and routes to handle_vital().

Threshold Check and Anomaly Detection

Off-body state is checked first (supreme rule). Then an Isolation Forest ML anomaly detector checks the reading against the patient's personal baseline, and hard thresholds are checked (HR>120, SpO₂<94, systolic BP>140, glucose>140 or <70). If either is true, priority=CRITICAL; otherwise priority=NORMAL.

Three-Tier Database Write (MySQL)

Tier	Name	What It Stores	Lifetime
Tier 1	Hot Buffer	Every edge-filtered packet — live WebSocket chart data, TTL index wipes automatically	24 hours — deleted at midnight
Tier 2	Warm Trend	Hourly baselines compressed at midnight from the day's edge-filtered points	7 days of hourly records

Tier	Name	What It Stores	Lifetime
Tier 3	Cold Ledger	One row per day — that day's final averaged health metrics	Full ~270 days — permanent clinical record

MySQL uses mysql-connector with %s positional placeholders (not asynpg's \$1/\$2), and NOW() / CURDATE() date functions. The query logic itself — insert into Hot Buffer, cron-compress to Warm Trend, cron-compress to Cold Ledger — is unchanged regardless of database engine.

7. The Cloud Cron Job — Preserving History While Disconnected

If a patient disconnects her device, her on-demand thread is killed to free memory — so no background worker is alive to write scheduled hourly Blood Pressure points or daily summaries during that gap. To fix this, long-interval metrics (BP hourly, daily summaries) are handled by a lightweight Cloud Cron / Scheduled Database Worker that runs independently of whether any patient's personal loop is active, wakes on schedule, commits a baseline reading, and goes back to sleep.

NOTE: On Render's free tier the server sleeps after 15 minutes of inactivity, which can stop the cron job from firing. Fix: use an external pinger like cron-job.org to keep the server awake on a schedule.

8. WebSocket Server — Real-Time Delivery

Flask-SocketIO maintains a persistent connection to every logged-in doctor's browser, using one private room per patient.

Doctor Authentication and Room Assignment

On connect, the JWT token is verified. When a doctor requests to watch a specific patient, a Zero Trust check queries patient_doctor_assignments — if he is not authorized for that patient, access is denied (403). If authorized, he joins room_{patient_id}.

Multi-Patient Isolation

Doctor A watching Patient P123 joins room_P123; Doctor B watching Patient P124 joins room_P124. When P123 data arrives, only Doctor A receives it — zero data leakage between patients or doctors.

Push Events

Normal readings emit a vitals_update event to the room. Critical readings emit a critical_alert event immediately, without waiting for the next cycle, and are also written to an alerts_log table.

9. Doctor Dashboard — The Frontend (React + Vite)

The browser maintains an active socket.io-client WebSocket connection to the server. Environment variables are read via import.meta.env.VITE_* (Vite's convention) rather than process.env.REACT_APP_*.

Three Dashboard UI Patterns

Pattern 1 — Exception-Based Alerts Inbox

- Full-screen red overlay flashes instantly on critical_alert
- Audio chime fires even if the browser is minimised
- Specific offending metric and value displayed prominently

- Alert logged to the Alerts Inbox with timestamp

Pattern 2 — Triage Patient Grid (Color-Coded)

Card Color	Meaning
Green	All vitals normal, metrics hidden in dropdown, screen stays clean
Red	High risk — row expands automatically showing the specific offending metric
Yellow	Sensor off-body — device powered on but not worn
Grey	Device offline / disconnected

Pattern 3 — On-Demand Trend Analytics

Clicking "View Analytics Chart" sends a GET request pulling from the correct database tier: range=24h from Hot Buffer, range=7d from Warm Trend, range=9m from Cold Ledger.

Partial History — The "Day 3" Problem

If a patient enrolled 3 days ago but a 7-day chart is requested, the chart pads the missing 4 days as "No Enrollment Data Available" (N/A) rather than crashing. A vertical "Device Inception / Onboarding Date" line marks where real data begins. Numerical averages are calculated as $\text{Sum of Vitals} \div \text{Total Enrolled Days (3, not 7)}$ — not divided by the full window, to avoid distorting the average. This same logic applies to any partial-history point in the pregnancy, not just Day 3, and both the patient's and doctor's dashboards display this identically since they read the same shared ledger.

10. Patient-Facing Vitals Views

Connecting and Disconnecting the Wearable

From My Wearable Device: clicking "Connect Wearable Device" shows a loading spinner ("Starting your health monitor..."), then the backend spawns the simulation thread and status becomes "● Connected — Live vitals active." Clicking "Disconnect Device" kills the thread; status becomes "○ Disconnected" with last known values greyed out, and the doctor's dashboard shows "Patient device offline."

Three Simultaneous Views

View	What It Shows	Updated How Often
Live readings	Current value of each parameter	Per-parameter interval
Daily vitals	Today's readings and daily summary	Throughout the day
Weekly averages	Average per parameter over last 7 days	Updated daily at midnight
Monthly averages	Average per parameter over last 30 days	Updated daily at midnight

11. Vitals Sharing Control — Per-Doctor Toggle

The wearable connection and the vitals sharing toggle are two completely separate controls. Connecting the wearable does NOT automatically share vitals with any doctor.

Control	What It Does	When It Is Set
Connect Wearable button	Starts simulation thread — patient sees her own vitals	Any time — independent

Control	What It Does	When It Is Set
Live vitals toggle per doctor	Controls whether that doctor sees the live WebSocket feed	Any time — per doctor
Disconnect from hospital	Removes all access for that doctor permanently	Patient's choice

Toggle ON: backend adds the doctor's socket to room_P123_DOCRAMESH; live vitals start flowing immediately. Toggle OFF: backend removes his socket from the room; his dashboard shows "Patient has paused live vitals sharing" while the patient's own view is unaffected.

- Rule 1 — Toggle affects live vitals only. Historical records are always visible to the doctor regardless of toggle state.
- Rule 2 — Doctor is notified when toggled off, as an informational status, not an alarm.
- Rule 3 — Patient can toggle any time. No hospital permission, no doctor approval needed.

If connected to multiple hospitals, each doctor has an independent toggle, with independent WebSocket rooms.

12. Self-Recording Before Hospital Connection

A patient can connect the wearable and collect vitals entirely independently, without any hospital or doctor connection — many women begin monitoring before their first hospital visit.

Self-recorded rows are stored with doctor_id=null, hospital_id=null, source="self" — completely private, visible only to the patient.

Sharing Decision on First Hospital Connection

When the patient later connects to a hospital and doctor, a one-time prompt asks what she wants to share:

Patient Chooses	What Doctor Sees
Yes — Share all past records	ALL vitals from day one, including self-recorded history (source="self" for pre-connection records)
No — Share only from today	Only vitals from the connection date onward; self-recorded history stays completely private

This decision is made entirely by the patient, never the hospital or doctor. A general (non-maternal) user can self-record in exactly the same way; only the LLM's response style and vitals normal ranges differ for them (covered in Phase 4).

13. Doctor-Facing Vitals Views

Clicking "View Vitals" on a patient record opens the same four sections the patient sees on her own dashboard: Live, Daily, Weekly, Monthly. The doctor can only view — never edit, delete, or trigger a reading — and whether his view is live at all is entirely controlled by the patient's per-doctor sharing toggle.

Edge Cases — Doctor Viewing Vitals

Edge Case	What Happens
Patient's live-sharing toggle is OFF for this doctor	Live section shows "Patient has paused live vitals sharing." Historical data recorded before the toggle was switched off remains fully visible.
Patient has never connected her wearable	All four sections show "No vitals recorded yet."
Patient disconnects her wearable device entirely	Live readings show "Device offline," last known values greyed out. Already-recorded aggregates remain visible.

Edge Case	What Happens
Wearable is placed on a table (off-body state)	Live readings go flat or show a gap with "Sensor Off-Body / No Signal" — never mistaken for a health crisis.
A genuine crisis spike occurs on a patient with the toggle ON	A red alarm banner appears instantly, the same moment the patient's own critical-vitals alert fires.

14. Complete End-to-End Flow Examples

A Normal, Non-Critical Reading

Simulator generates HR 82 → Deadband check suppresses if change < 10 → 5-minute heartbeat eventually force-transmits → MQTT publish (QoS 1) → Mosquitto routes to Flask (wildcard subscription) → threshold check finds it non-critical → written to Hot Buffer → WebSocket emits vitals_update to room_P123 → doctor's chart updates, no alert.

HR Spikes to 135 bpm (Crisis)

Deadband and danger checks both trigger immediate transmit at QoS 2, on topic patients/P123/alert/critical → Flask routes directly to the alert pipeline, skipping the normal flow → written to Hot Buffer with priority=CRITICAL and to alerts_log → WebSocket emits critical_alert immediately → doctor's browser flashes a red banner, plays an audio alert, shows an overlay modal, and turns the patient grid card red.

Phase 3 Summary Table

Component	Key Rule Established
Simulation engine	Python script per patient, spawned on Connect, killed only on Disconnect (not Logout).
4-Quadrant state machine	Table state is supreme; manual overrides beat autonomous dice rolls.
Edge layer	Deadband filter, moving window average, daily aggregation, 5-minute heartbeat.
Transport	MQTT via Mosquitto, QoS 1 normal / QoS 2 critical.
Storage	MySQL, three tiers — Hot Buffer (24h), Warm Trend (7d), Cold Ledger (~270 days).
Real-time delivery	Flask-SocketIO, one room per patient, Zero Trust authorization check.
Sharing control	Per-doctor toggle, independent of wearable connection and hospital connection.
Self-recording	Fully supported before any hospital connection; patient controls what to share later.